

SKILL EXPERIMENT 20

NAME: K ANUSRI

ROLL NO: 2520030432

Title

Implementation of SIGTSTP Handling and Process Group Management

Aim

To capture SIGTSTP, suspend processes, update the job table, preserve process state, resume jobs later, and test the suspension workflow.

To create process groups, assign group IDs, transfer terminal control, restore terminal ownership, coordinate signals, and validate process group behavior.

Theory

SIGTSTP is a terminal-generated signal used to suspend a running foreground process. When a process is suspended, its state is recorded in the job table so that it can be managed and resumed later.

Process groups provide a way to manage related processes as a single unit. Each process group is identified by a group ID. Terminal control can be transferred to the appropriate process group when a job is running in the foreground. After the job stops or completes, terminal ownership is restored to the shell.

Signals can be coordinated across a process group so that related processes receive the required signal together. Proper process group and terminal management ensures that job states remain consistent and the shell continues to operate correctly.

Algorithm

Part A – SIGTSTP and Job Suspension

1. Start the program.
2. Initialize the job table.
3. Register the required signal handling mechanism.
4. Identify the current foreground process.
5. Capture the SIGTSTP signal from the terminal.
6. Suspend the foreground process.
7. Identify the job ID of the suspended process.
8. Update the job state to stopped.
9. Preserve the process information in the job table.
10. Display the suspended job and its current state.

11. Resume the stopped job when required.
12. Update the job state after resumption.
13. Verify that the process continues execution.
14. Test the complete suspension and resume workflow.
15. Stop the program.

Part B – Process Groups and Terminal Control

1. Start the process group management operation.
2. Create a new process group for the job.
3. Assign and store the process group ID.
4. Add the required processes to the process group.
5. Identify the foreground process group.
6. Transfer terminal control to the selected process group.
7. Coordinate required signals for the process group.
8. Monitor the state of the processes.
9. Detect when the foreground job is suspended or completed.
10. Restore terminal ownership to the shell.
11. Update the corresponding job information.
12. Validate the process group and terminal control behavior.
13. Display the final process and job states.
14. Stop the program.

CODE



k_anusri@DESKTOP-TM20BAC



```
k_anusri@DESKTOP-TM20BAQ:~$  
k_anusri@DESKTOP-TM20BAQ:~$ #include <stdio.h>  
#include <unistd.h>  
#include <signal.h>  
#include <sys/wait.h>  
  
typedef struct {  
    int job_id;  
    pid_t pid;  
    pid_t pgid;  
    char state[20];  
} Job;  
  
Job job;  
  
void handle_sigtstp(int sig) {  
    printf("\nSIGTSTP received.\n");  
    printf("Job %d is being suspended...\n", job.job_id);  
    sprintf(job.state, "Stopped");  
}  
  
int main() {  
    pid_t pid;  
    int status;  
  
    job.job_id = 1;  
  
    signal(SIGTSTP, handle_sigtstp);  
  
    pid = fork();  
  
    if (pid == -1) {  
        perror("fork");  
        return 1;  
    }  
  
    if (pid == 0) {  
        setpgid(0, 0);  
  
        printf("Process Group Created.\n");  
        printf("PID = %d\n", getpid());  
        printf("PGID = %d\n", getpgrp());  
  
        printf("Process is running...\n");  
        sleep(2);  
    }  
}
```

```
        raise(SIGSTOP);

        printf("Process resumed successfully.\n");
        printf("Process completed.\n");
        return 0;
    }

    setpgid(pid, pid);

    job.pid = pid;
    job.pgid = pid;
    sprintf(job.state, "Running");

    printf("\nJob Table:\n");
    printf("Job ID: %d | PID: %d | PGID: %d | State: %s\n",
           job.job_id, job.pid, job.pgid, job.state);

    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status)) {
        sprintf(job.state, "Stopped");

        printf("\nJob suspended.\n");
        printf("Job Table Updated:\n");
        printf("Job ID: %d | PID: %d | PGID: %d | State: %s\n",
               job.job_id, job.pid, job.pgid, job.state);

        printf("\nSending SIGCONT to process group...\n");
        kill(-job.pgid, SIGCONT);

        sprintf(job.state, "Running");
        printf("Job resumed. State: %s\n", job.state);
    }

    waitpid(pid, &status, 0);

    sprintf(job.state, "Completed");

    printf("\nTerminal ownership restored.\n");
    printf("Final Job State: %s\n", job.state);
    printf("Process group and signal test completed successfully.\n");

    return 0;
}
```

COMPILATION

```
hp@DESKTOP-35T9S8P:~$ nano exp5.c
hp@DESKTOP-35T9S8P:~$ gcc exp5.c -o exp5
hp@DESKTOP-35T9S8P:~$ ./exp5

Job Table:
Process Group Created.
Job ID: 1 | PID: 620 | PGID: 620 | State: Running
PID = 620
PGID = 620
Process is running...

Job suspended.
Job Table Updated:
Job ID: 1 | PID: 620 | PGID: 620 | State: Stopped

Sending SIGCONT to process group...
Job resumed. State: Running
Process resumed successfully.
Process completed.

Terminal ownership restored.
Final Job State: Completed
Process group and signal test completed successfully.
hp@DESKTOP-35T9S8P:~$ |
```

Result

SIGTSTP was captured successfully and the foreground process was suspended while preserving its state in the job table. The stopped job was later resumed and its status was updated. Process groups were created and assigned group IDs, terminal control was transferred and restored, and signal coordination was validated successfully.